

Lab 5: Seven-Segment Display

Purpose

The purpose of this lab is to use multiple seven-segment displays on BASYS 3 board and display numbers simultaneously.

Questions

- 1- What is the internal clock frequency of Basys 3?

Internal clock of the BASYS 3 is 100 MHz. In the design, this clock will be used and adjusted according to the needs.

- 2- How can you create a slower clock signal from this one?

Clock divider is a useful way to adjust clock frequency. To illustrate, while using BASYS 3 whose internal clock is 100 MHz, value can be toggled in every 2 falling/rising edges of the clock. Therefore, one can obtain a clock having a 50 MHz which is the half of the base clock.

- 3- Can you create a clock with any arbitrary frequency lower than that of the internal clock? If not, which frequencies can you create?

Since the methodology is taking action during specific behavior of the internal clock, the possible divided clock frequencies are limited, they cannot be generated arbitrarily. Only integer values who are the divisors of the base clock can be created by using clock division method.

Design Specifications

Each seven-segment display consists of 7 distinct light emitting diodes (LED). These are together allow to display numbers from 0 to 9. They have two inputs: anode and cathode. one is anode and the other is cathode. The BASYS 3 board contains a four-digit common anode seven-segment LED display.

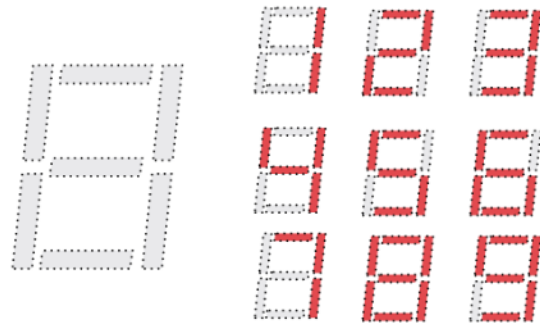


Figure 1: Representation of numbers in seven-segment display

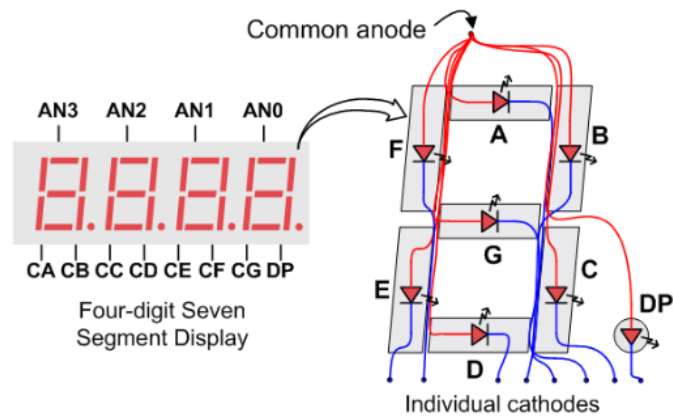


Figure 2: Seven-segment display connections

There are 4 segments in BASYS 3; however, they cannot be high at the same time. Segments are becoming high and low at a remarkably high speed. Human eye cannot catch the changes in high and low values. Therefore, it is likely to seem as more than one segment is emitting light at the same time. This is called as “persistence of vision”. This can be proven by the photos in the appendices. On the photos, some of the segments are low, but they do not seem so. Additionally, in photos there are examples that including more than one segment is high, this is so because, during capture time, there are many transitions made from high to low or low to high. Since capture time is much longer compared to time that a segment is active, it is indeterminate that which segments will be captured as low or high. The numbers are too small so in the photos different number of segments can be captured as high.

For the implementation of the seven-segment display a main module named as “main.vhd” will be written. It will have 2 inputs, clk and reset, and 2 outputs, anode and cathode. Clk is the internal clock of the BASYS 3 having 100MHz frequency and reset will be controlled by users externally, therefore, it will be assigned to the rightmost switch V17. Anode and cathode are not controlled by users, they are internal structures and will be assigned U2, U4, V4, W4 and W7, W6, U8, V8, U5, V5 and U7, respectively.

To avoid complexity, modular fashion will be used and therefore, 2 distinct modules will be used. These will be called as “clockController.vhd” and “segmentController.vhd”

Methodology

In this lab, a hexadecimal counter will be implemented and displayed on the seven-segment display component on BASYS3. The counter starts from 0 and by incrementing displayed number 1, it goes up. As long as the program works steady the counter keeps counting upwards. However, when the reset switch is turned high, count becomes 0 and when it is low again, counter starts counting again from 0.

In the module “clockController”, clock is adjusted. And, in the module “segmentController”, segments are activated in the first process. This is done by checking the signal “converter” which is a 2-bit binary number and coming from the main module. In the second process, the LED’s will be turned on are declared according to the hexadecimal value of the binary signal names as “binaryNumber”. Lastly, in the main module, signals are declared and two modules are used.

Constraint file consists of internal structures which is different than previous labs. Instead of switches and LEDs, internal clock signal and internal anode/cathode assignments are made. Clock is assigned to internal port W5. Anode is assigned to U2, U4, V4, W4 for its range 0 to 3, respectively. Likewise, cathode is assigned to U7, V5, U5, V8, U8, W6, W7 for its range 0 to 6. In this design, only thing controlled externally is reset and it is assigned to the rightmost switch V17.

Test bench file includes the main module as component and two processes, clock process and stimulus. By using this, the waveform graph given in the results part.

Results

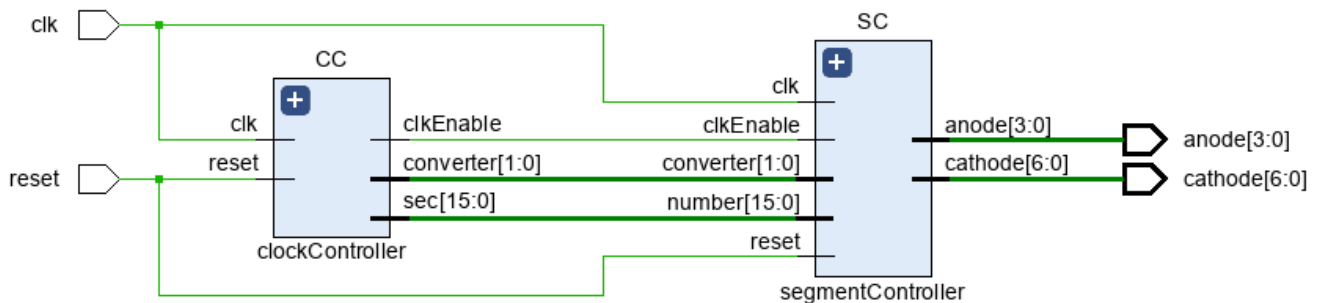


Figure 3: Schematic of the main module of the seven-segment display

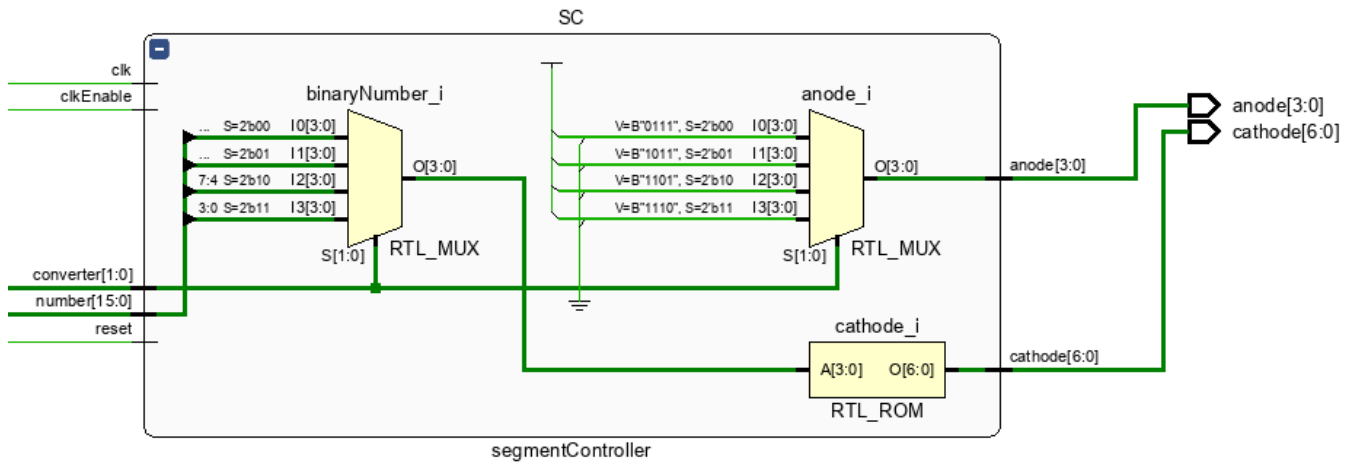


Figure 5: Schematic of the segmentController module

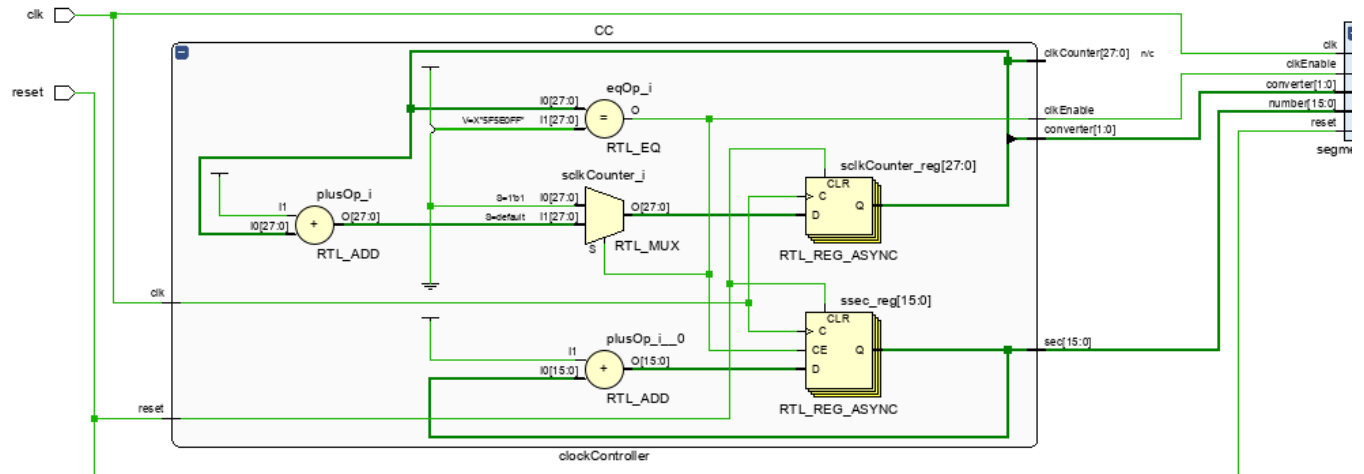


Figure 4: Schematic of the clockController module

To test the code before running it on the BASYS 3 board, test bench is written. Waveform graph shows the transitions in clock and therefore in anode and cathode. It matches with the expectations. The example photos are given in the appendices.

Appendices

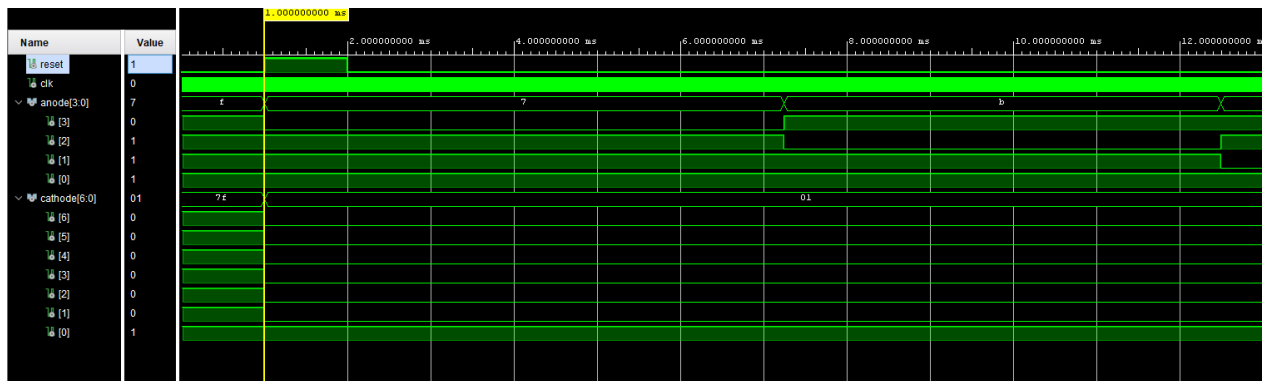


Figure 7: Waveform graph showing the transitions when and after turning reset on

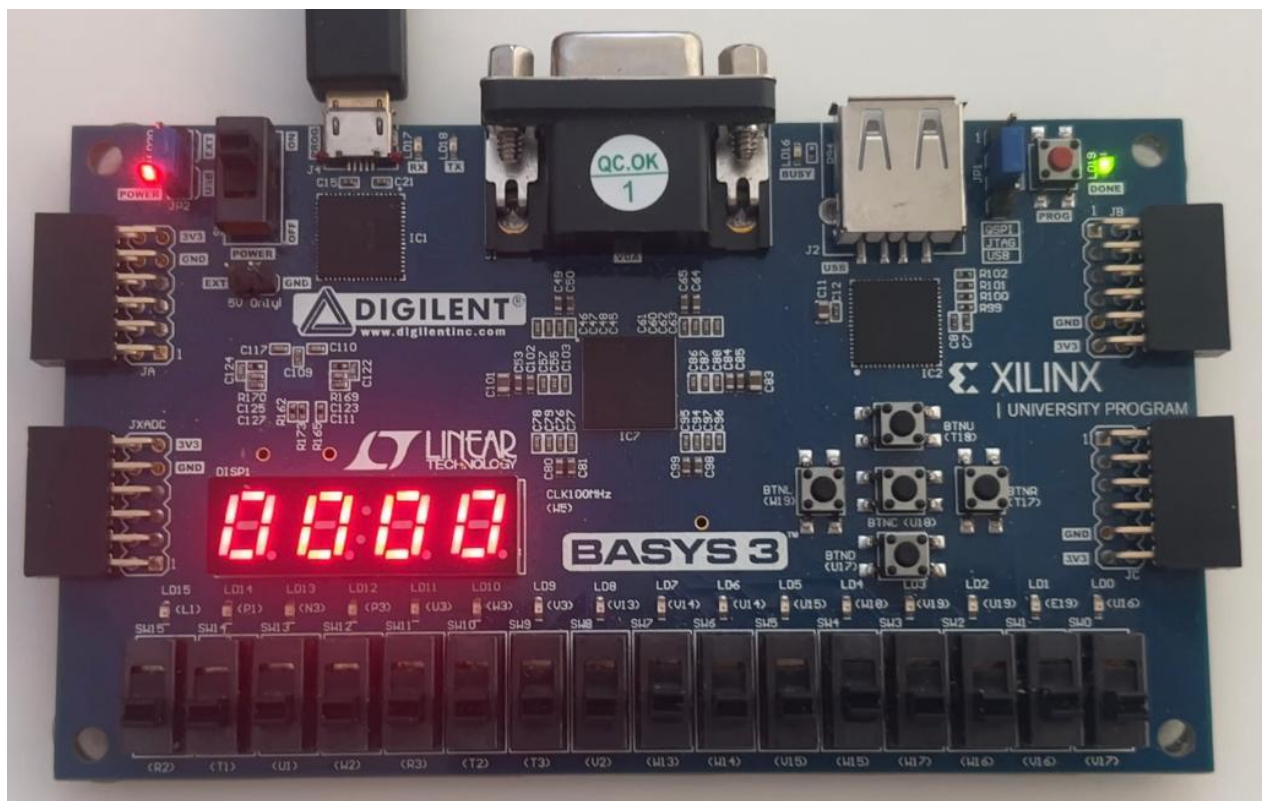


Figure 8: Initial display right after BASYS 3 is programmed

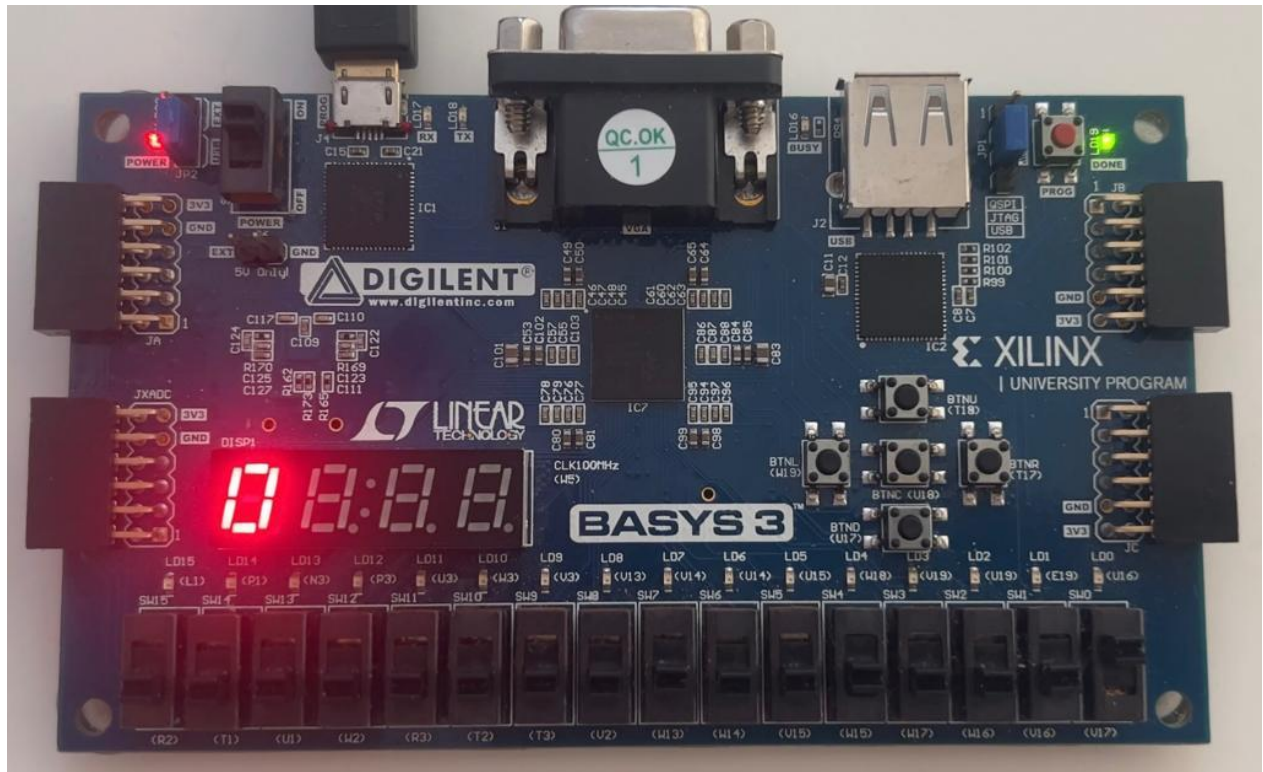


Figure 9: Seven-segment display when reset switch is high

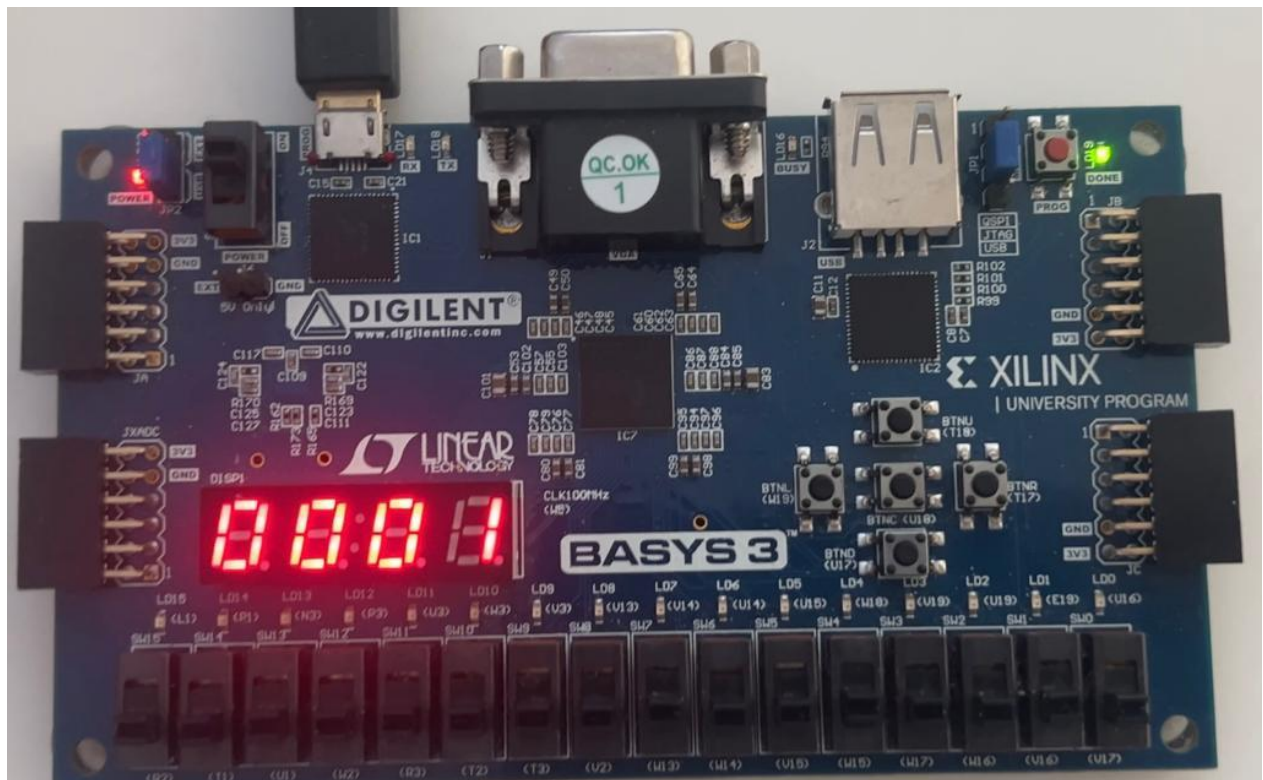


Figure 10: Seven-segment display at second 1

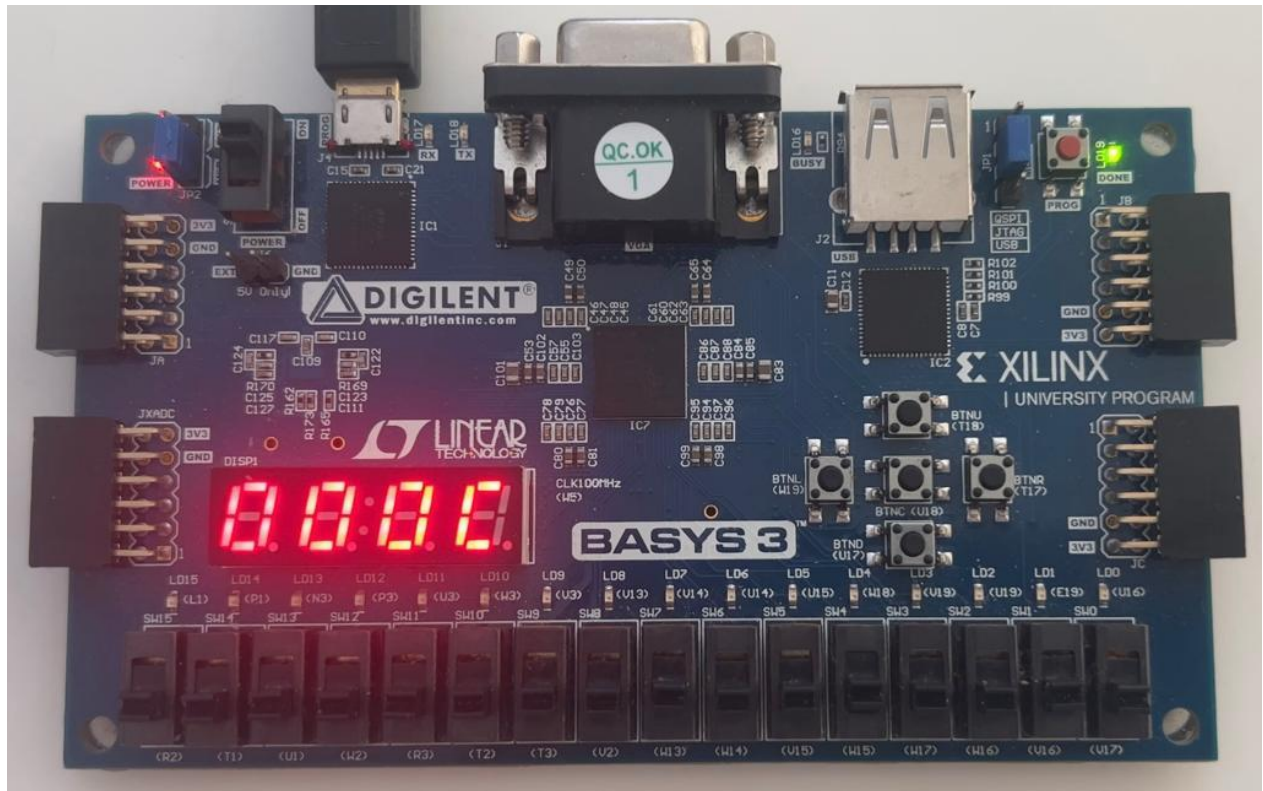


Figure 11: Seven-segment display at second 12, 12's hexadecimal value C is displayed

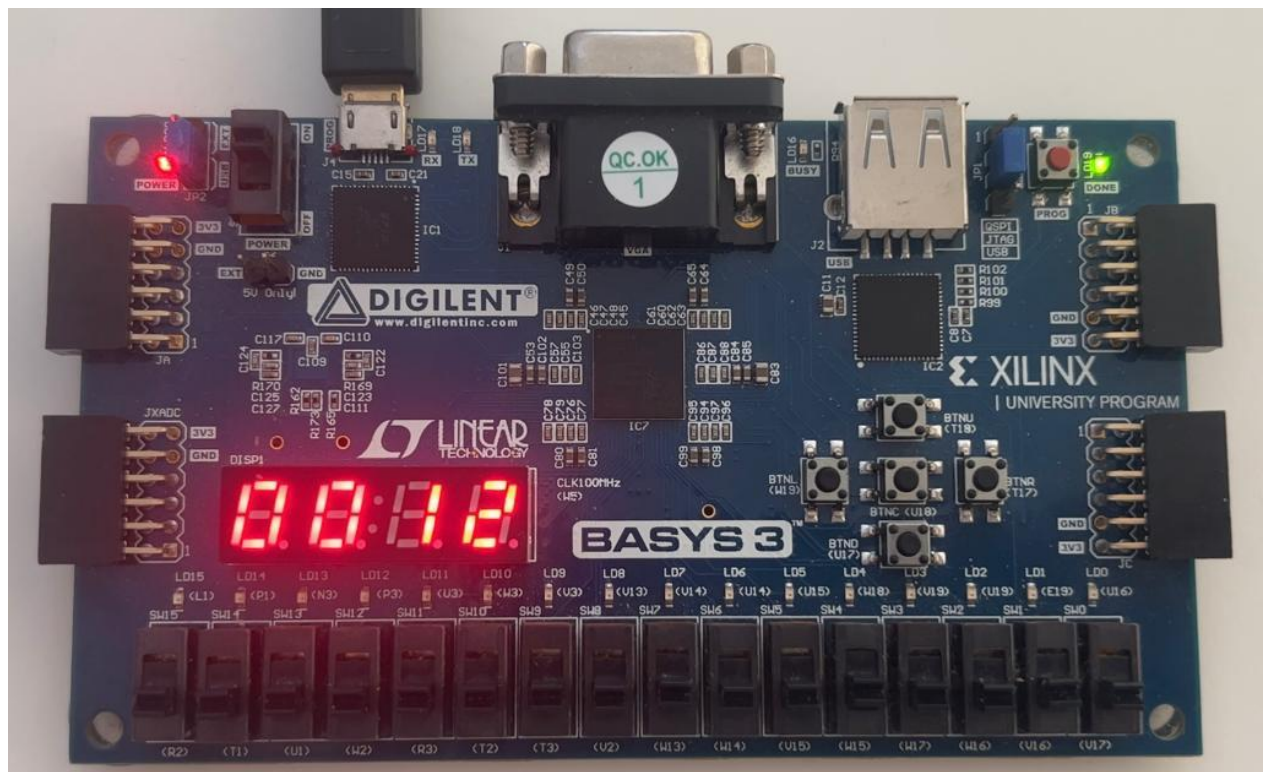


Figure 12: Seven-segment display at second 18

```
library IEEE;
use IEEE.NUMERIC_STD;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

entity main is

```
    Port (clk    : in std_logic;
          reset   : in std_logic;
          anode   : out std_logic_vector(3 downto 0);
          cathode : out std_logic_vector(6 downto 0));
```

end main;

architecture Behavioral of main is

```
    signal clkEnable : std_logic;
    signal converter : std_logic_vector(1 downto 0);
    signal number    : std_logic_vector(15 downto 0);
```

component clockController is

```
    Port (clk    : in std_logic;
          reset   : in std_logic;
          clkEnable : out std_logic;
          converter : out std_logic_vector(1 downto 0);
          sec      : out std_logic_vector(15 downto 0);
          clkCounter : out std_logic_vector(27 downto 0));
```

end component;

component segmentController is

```
    Port (clk    : in std_logic;
```

```

    reset    : in std_logic;
    clkEnable : in std_logic;
    converter : in std_logic_vector(1 downto 0);
    number    : in std_logic_vector(15 downto 0);
    anode     : out std_logic_vector(3 downto 0);
    cathode   : out std_logic_vector(6 downto 0));
end component;

```

```
begin
```

```

    CC: clockController port map(clk => clk, reset => reset, clkEnable => clkEnable, converter =>
converter, sec => number);

```

```

    SC: segmentController port map(clk => clk, reset => reset, clkEnable => clkEnable, converter
=> converter, number => number, anode => anode, cathode => cathode);

```

```
end Behavioral;
```

Code 1: Main module

```

library IEEE;
use IEEE.NUMERIC_STD;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```
entity clockController is
```

```

    Port (clk      : in std_logic;
          reset    : in std_logic;
          clkEnable : out std_logic;
          converter : out std_logic_vector(1 downto 0);
          sec      : out std_logic_vector(15 downto 0);

```

```
        clkCounter : out std_logic_vector(27 downto 0));  
end clockController;
```

architecture Behavioral of clockController is

```
    signal ssec      : std_logic_vector(15 downto 0) := (others => '0');  
    signal sclkCounter : std_logic_vector(27 downto 0);
```

```
begin
```

```
    process(clk) begin
```

```
        if(reset = '1') then
```

```
            sclkCounter <= (others => '0');
```

```
            ssec <= (others => '0');
```

```
        else
```

```
            if rising_edge(CLK) then
```

```
                sclkCounter <= sclkCounter + '1';
```

```
                if sclkCounter = x"5F5E0FF" then
```

```
                    ssec <= ssec + '1';
```

```
                    sclkCounter <= (others => '0');
```

```
                end if;
```

```
            end if;
```

```
        end if;
```

```
    end process;
```

```
    clkEnable <= '1' when sclkCounter = x"5F5E0FF" else '0';
```

```
    converter <= sclkCounter(19 downto 18);
```

```
    clkCounter <= sclkCounter;
```

```
    sec <= ssec;
end Behavioral;
```

Code 2: clockController module

```
library IEEE;
use IEEE.NUMERIC_STD;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity segmentController is
    Port (clk      : in std_logic;
          reset    : in std_logic;
          clkEnable : in std_logic;
          converter : in std_logic_vector(1 downto 0);
          number    : in std_logic_vector(15 downto 0);
          anode     : out std_logic_vector(3 downto 0);
          cathode   : out std_logic_vector(6 downto 0));
end segmentController;

architecture Behavioral of segmentController is
    signal binaryNumber : std_logic_vector(3 downto 0);
begin

    process(converter) begin
        case converter is
            when "00" => anode <= "0111"; -- activates first segment
            binaryNumber <= number(15 downto 12);
            when "01" => anode <= "1011"; -- activates second segment
```

```
binaryNumber <= number(11 downto 8);  
when "10" => anode <= "1101"; -- activates third segment  
binaryNumber <= number(7 downto 4);  
when "11" => anode <= "1110"; -- activates fourth segment  
binaryNumber <= number(3 downto 0);  
when others => anode <= "1111";  
end case;  
end process;
```

```
process(binaryNumber) begin  
  case binaryNumber is  
    when "0000" => cathode <= "0000001"; -- 0  
    when "0001" => cathode <= "1001111"; -- 1  
    when "0010" => cathode <= "0010010"; -- 2  
    when "0011" => cathode <= "0000110"; -- 3  
    when "0100" => cathode <= "1001100"; -- 4  
    when "0101" => cathode <= "0100100"; -- 5  
    when "0110" => cathode <= "0100000"; -- 6  
    when "0111" => cathode <= "0001111"; -- 7  
    when "1000" => cathode <= "0000000"; -- 8  
    when "1001" => cathode <= "0000100"; -- 9  
    when "1010" => cathode <= "0000010"; -- a  
    when "1011" => cathode <= "1100000"; -- b  
    when "1100" => cathode <= "0110001"; -- c  
    when "1101" => cathode <= "1000010"; -- d  
    when "1110" => cathode <= "0110000"; -- e  
    when "1111" => cathode <= "0111000"; -- f  
    when others => cathode <= "1111111";  
  end case;  
end process;
```

```
end process;  
end Behavioral;
```

Code 3: segmentController module

```
# Clock signal  
set_property PACKAGE_PIN W5 [get_ports clk]  
set_property IOSTANDARD LVCMOS33 [get_ports clk]  
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]  
  
#seven-segment LED display  
set_property PACKAGE_PIN V17 [get_ports {reset}]  
set_property IOSTANDARD LVCMOS33 [get_ports {reset}]  
set_property PACKAGE_PIN W7 [get_ports {cathode[6]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[6]}]  
set_property PACKAGE_PIN W6 [get_ports {cathode[5]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[5]}]  
set_property PACKAGE_PIN U8 [get_ports {cathode[4]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[4]}]  
set_property PACKAGE_PIN V8 [get_ports {cathode[3]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[3]}]  
set_property PACKAGE_PIN U5 [get_ports {cathode[2]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[2]}]  
set_property PACKAGE_PIN V5 [get_ports {cathode[1]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[1]}]  
set_property PACKAGE_PIN U7 [get_ports {cathode[0]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {cathode[0]}]  
set_property PACKAGE_PIN U2 [get_ports {anode[0]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {anode[0]}]  
set_property PACKAGE_PIN U4 [get_ports {anode[1]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {anode[1]}]
```



```

set_property PACKAGE_PIN V4 [get_ports {anode[2]]}

    set_property IOSTANDARD LVCMOS33 [get_ports {anode[2]]}
set_property PACKAGE_PIN W4 [get_ports {anode[3]]}

    set_property IOSTANDARD LVCMOS33 [get_ports {anode[3]]}

```

Code 4: Constraint file

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity testBench is
-- Port ( );
end testBench;

architecture Behavioral of testBench is

    signal reset  : std_logic;
    signal clk    : std_logic;
    signal anode  : std_logic_vector(3 downto 0);
    signal cathode : std_logic_vector(6 downto 0);

    component main is
        Port (clk    : in std_logic;
              reset  : in std_logic;
              anode  : out std_logic_vector(3 downto 0);
              cathode : out std_logic_vector(6 downto 0));
    end component;

begin
    uut: main port map(clk => clk, reset => reset, anode => anode, cathode => cathode);


    clockProcess: process begin
        clk <= '0';
        wait for 10 ns;
        clk <= '1';
        wait for 10 ns;
    end process;

    stimulusProcess: process begin

```

```
    reset <= '0';  
    wait for 1 ms;  
    reset <= '1';  
    wait for 1 ms;  
    reset <= '0';  
    wait;  
end process;
```

```
end Behavioral;
```

Code 5: Test bench

References

<https://digilent.com/reference/programmable-logic/basys-3/reference-manual>